QwenPI VLA Training Optimization on 8×H200

A profile-driven journey from 800 ms to 249 ms per step (3.2×)

Qihan Kang | NVIDIA | July 2026

Results at a Glance

starVLA QwenPI: Qwen3.5-VL 4B + flow-matching action head, ZeRO-2 full fine-tune

3.2×

Faster training step

800 → 249 ms/step (bs 8)

257

Samples / node / s

up from 80 at hand-off

23.1%

Model FLOPs utilization

up from 7.2% (bf16 peak)

340

Samples / node / s

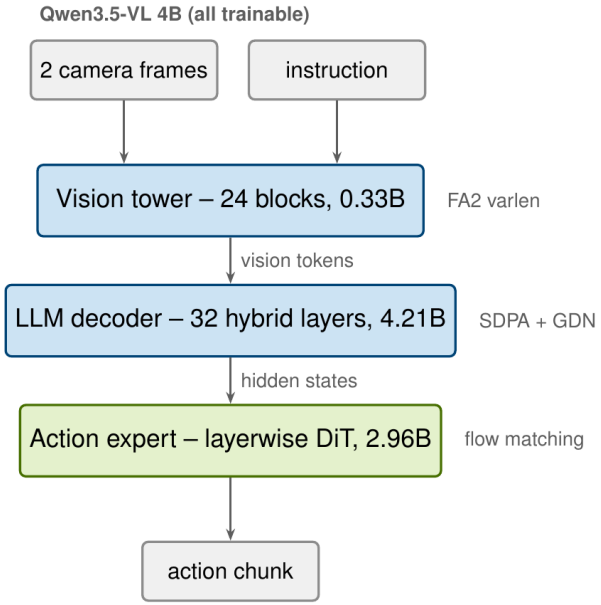
at bs 24 — 1.7× the 200 target

Method: every optimization was profiled with Nsight Systems and validated with a same-node A/B run plus loss-trajectory checks — no stacked guesses, measured deltas only.

Measurement: 8×H200 (NVLink), 60 steps, steady-state p50 of steps 21–44, per-GPU batch 8 unless noted.

Model & Workload

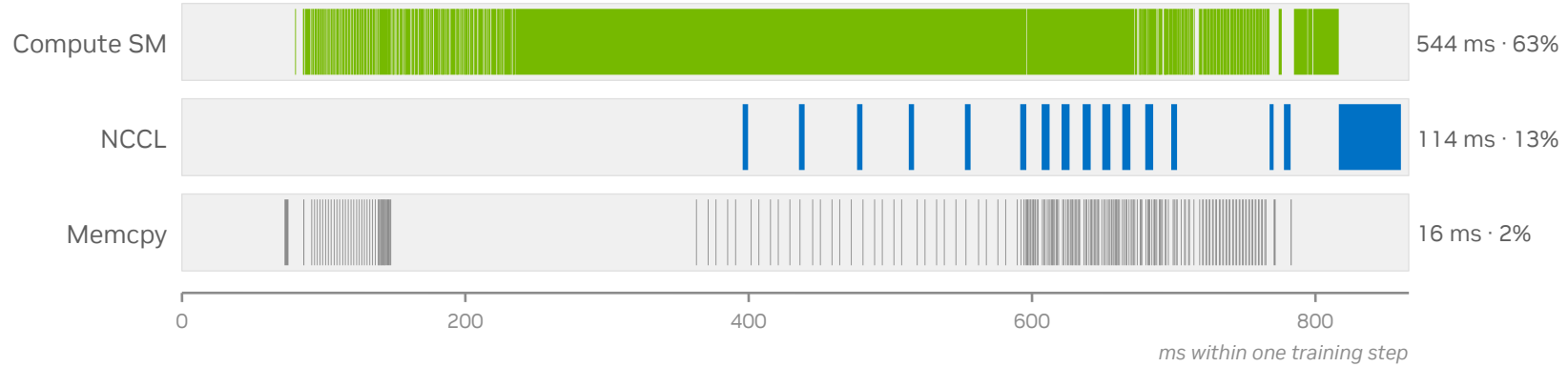
7.5B trainable parameters: Qwen3.5-VL 4B backbone + 3B flow-matching action expert



Training configuration	
Parameters	7.52B, all trainable (bf16)
Parallelism	DeepSpeed ZeRO-2, DP=8, 1 node
ZeRO-2 tuning	bucket 1.5e9, reduce_scatter off
Optimizer	AdamW, fp32 master (sharded)
Batch	8 / GPU → 64 global (up to 24/GPU)
Sequence	text fixed at 192 (left-pad)
Data	LIBERO manipulation, 2 views/sample
Hardware	8×H200 SXM, full NVLink
Software	torch 2.7.1 / NCCL 2.26.2 / HF 5.3 / FA2.8

Baseline Diagnosis: Where Did 800 ms Go?

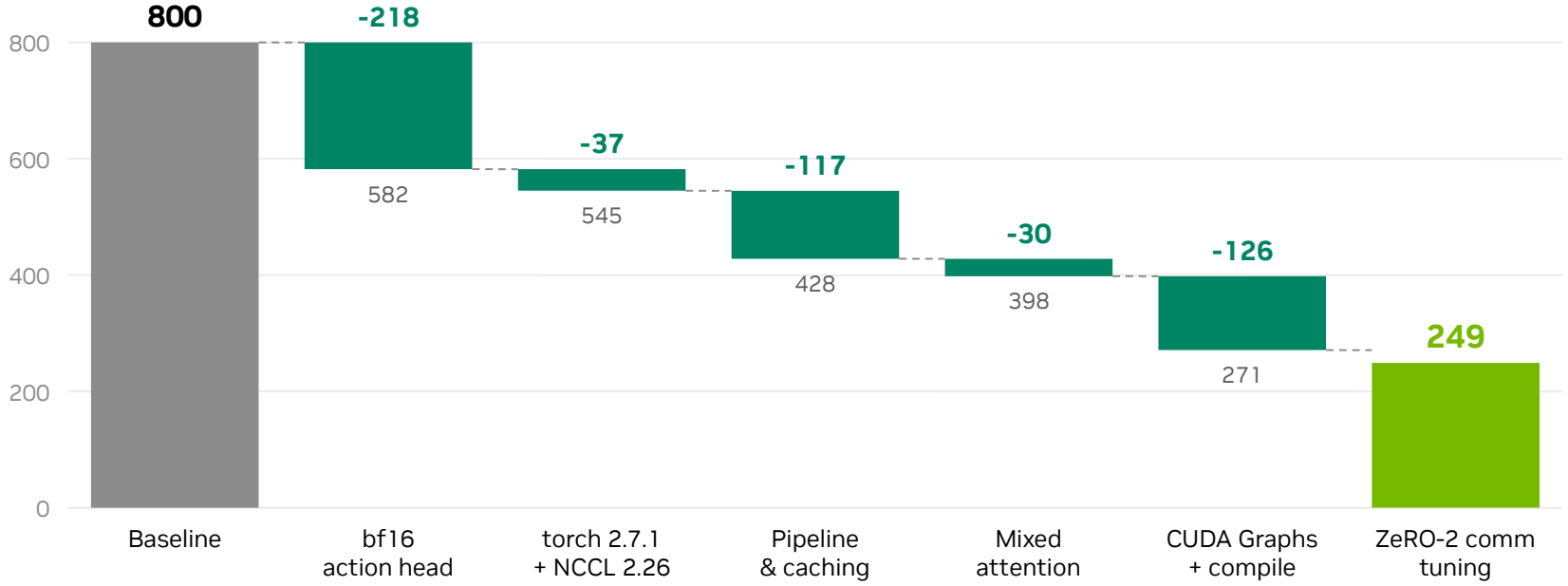
Real per-stream activity of one baseline step (nsys sqlite, 866 ms wall)



- **29% pure GPU idle —** 254 ms with no kernel on any stream — the GPU waits for the CPU
- **13,868 kernel launches / step —** eager launch overhead dominates; many kernels run < 20 µs
- **Action head runs in fp32 —** the 2.96B DiT sits outside autocast — half the achievable math rate
- **CPU work inside forward() —** HF preprocessing + per-step syncs serialize CPU and GPU

Optimization Roadmap

Six measured steps, ms per step — each level profiled on the same node



Ladder levels measured with nsys attached on rank 0 (~5% overhead); final 249 ms is the clean steady-state p50 of the merged main branch (bucket 1.5e9 + reduce_scatter off).

First Wins: bf16 Head + Software Stack

800 → 545 ms. The two cheapest changes bought 32%

bf16 action head

−218 ms (800 → 582)

- The 2.96B DiT ran in fp32 — one config knob moves it under bf16 autocast
- Loss & timestep math stay fp32; same-seed 100-step loss trajectory unchanged
- Single-switch rollback: dit_dtype=none

torch 2.7.1 + NCCL 2.26.2

−37 ms (582 → 545)

- Container-level upgrade, zero code change (plus triton 3.3.1)
- Faster eager kernels and NCCL launch path
- Prerequisite for the CUDA-Graph work (slide 8)

Takeaway: audit precision policy and framework versions before touching any code — they set the floor everything else stands on.

Keep the GPU Fed: Pipeline & Syncs

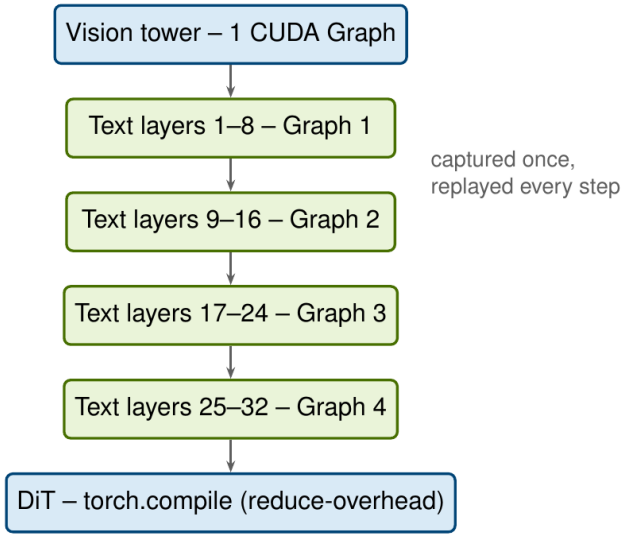
545 → 398 ms. CPU work off the critical path, no per-step syncs, right attention per stack

Preprocess in DataLoader workers	HF processor (tokenize + image patches) runs in 8 workers, overlapped with the previous step's GPU work; batches arrive as pinned CPU tensors	**~40 ms of forward-path CPU removed**
Side-stream H2D copies	Pinned host batch + dedicated copy stream overlaps upload with the ZeRO-2 optimizer tail	**H2D off the critical path**
Sync-free logging	DeepSpeed timers unsynchronized; loss .item() only on logging steps	**removes 2 device syncs / step**
MRoPE position-id cache	3D position ids precomputed on CPU and cached — they depend only on the image grid	**~13 ms sync eliminated**
Mixed attention	Vision tower on FA2 varlen (one kernel per layer for all 16 images); text stays SDPA — at seq 192 FA2's unpad/repad is a net loss (measured)	**−30 ms**

Principle: a training step should launch GPU work and get out of the way — anything the CPU must compute belongs in workers or caches, and anything that reads a GPU value blocks the launch pipeline.

CUDA Graphs & torch.compile: −126 ms

398 → 271 ms. Replay the whole network as six graphs instead of ~14k eager launches



- **Whole-model compile is blocked: the hybrid GDN layers (fla library) graph-break in dynamo**
- So we capture around it: 32 text layers grouped into 4 CUDA Graphs of 8 layers each
- Vision tower captured as a single graph (FA2 kernels inside, host-cached max_seqlen)
- Static shapes make it possible: text fixed at 192, encoder padded to 192
- Sync-free multimodal merge: removed a device-to-host validation readback in the image-token scatter

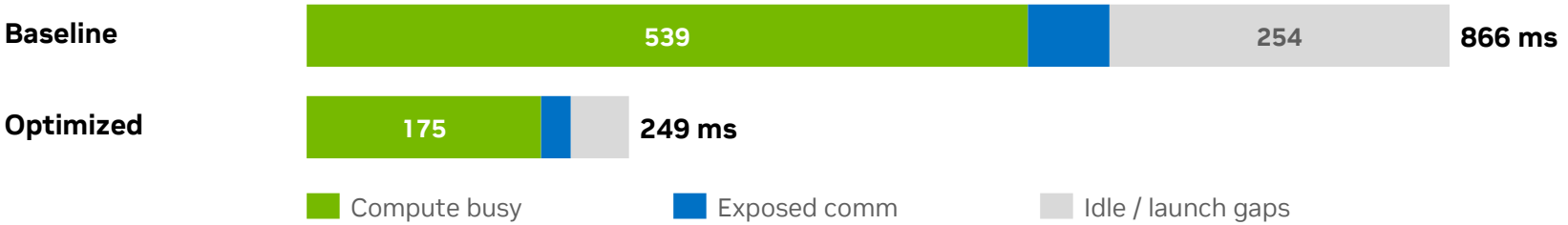
Kernel launches: 13.9k → 6.4k per step. CPU launch cost leaves the critical path; GPU idle collapses.

ZeRO-2 Comm Tuning & the Final Ledger

271 → 249 ms. Fewer, larger collectives — and an honest ledger of what remains

- reduce_bucket_size 5e8 → 1.5e9: 6 gradient AllReduces per step instead of ~20, scoped to this model's config only
- reduce_scatter=false: drops DeepSpeed's redundant per-bucket flatten (CatArrayBatchedCopy) ahead of each AllReduce
- Verified NOT bandwidth-bound: AllReduce microbenchmark hits 478 GB/s bus bandwidth — the NVLink per-direction ceiling. The remaining exposed comm can only shrink by sending fewer bytes.

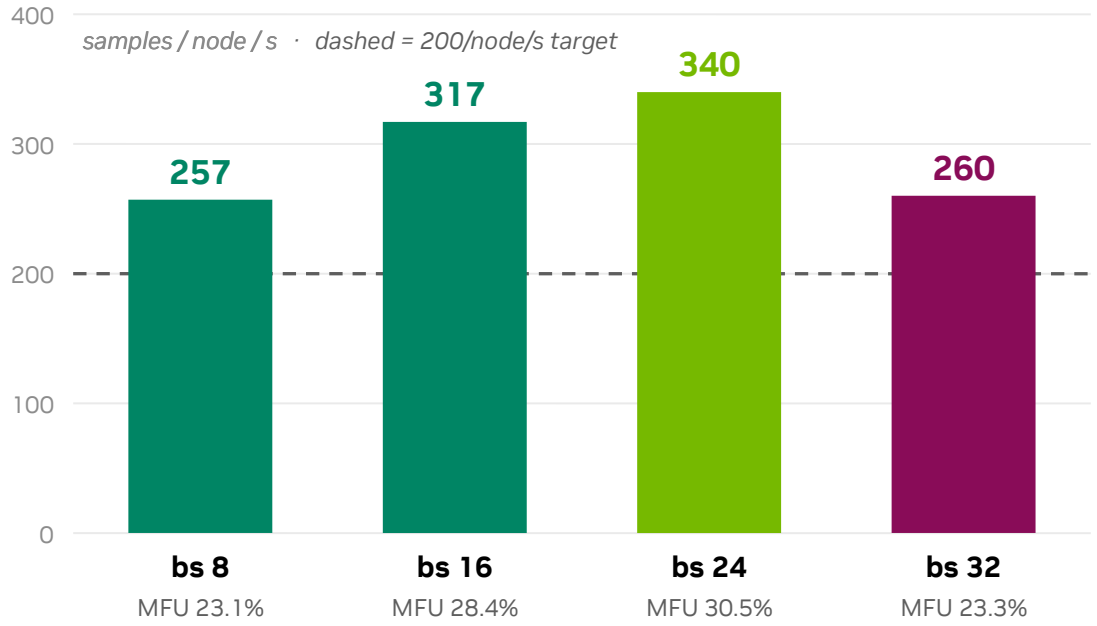
Per-step time ledger (same accounting, one steady step from each nsys capture)



Gradient AllReduce 15.1 GB/step + optimizer AllGather 15.1 GB/step (bf16). Idle from 254 ms to 43 ms.

Batch Scaling: Sweet Spot at 24 per GPU

Same optimized config, only per-GPU batch changes — peak 340 samples/node/s



- Batching amortizes launch and comm costs: MFU climbs 23% → 30.5%
- bs 32 regresses hard (986 ms/step): not OOM, not the dataloader — memory-pressure effects under investigation
- Larger global batch (192) changes optimization dynamics — needs a convergence run before adoption

What We Ruled Out — and How

Negative results are results: every hypothesis got a same-node experiment

Hypothesis	Experiment	Verdict
FA2 for the text stack	A/B at seq 192	Slower (+10 ms): unpad/repad overhead exceeds kernel gains at short sequences
“NCCL is misconfigured”	Forced protocols, NCCL_DEBUG, microbenchmark	AllReduce already at 478 GB/s busbw = NVLink ceiling; kernel-name suffix was a red herring
masked_scatter nonzero sync	index_copy patch, bitwise-equal unit test	0 ms: the AllReduce tail is gradient-readiness-bound, so hiding the sync moves nothing
MRoPE cache still needed?	Toggle at final config	Neutral at the tip (CUDA Graphs hide it) — kept for CPU headroom, one-knob rollback
Whole-model torch.compile	Compile attempts on torch 2.6 & 2.7	Hybrid GDN layers graph-break in dynamo → group-graph capture instead
FlashQLA GDN kernels	Kernel-time ceiling analysis	Only ~10.5 ms/step of GDN time exists; needs torch $\geq$ 2.8 — deferred, not worth the risk now

Remaining Headroom & Next Steps

Where the next milliseconds live, ranked by ceiling — measured, not guessed

Backward GEMMs at the power wall **[fp8 territory]**

Sustained GEMM throughput caps at ~622 TFLOPS on this node (power, not occupancy). The only lever left for the math itself is fp8.

Exposed communication (~30-55 ms) **[compression]**

Bandwidth is at the NVLink ceiling; bytes are the only lever — 1-bit Adam / gradient compression, with convergence sign-off.

Optimizer tail (~25 ms) **[compiled autograd]**

ZeRO-2 optimizer step + AllGather at step end; compiled autograd / fused optimizer can overlap part of it.

Scale-out readiness **[deployment]**

On split-topology nodes (4+4 NVLink islands) set NCCL_P2P_LEVEL=NVL — first collective hangs otherwise. bs24 adoption gated on a convergence run.